

LECTURE 20

Advanced Normalization: Functional Dependencies and BCNF

A formal proof system for FDs, and the normal form that catches what 3NF misses

CSC270 · Database Systems

COMSATS University Islamabad, Lahore Campus

From Axioms to the Strictest Normal Form

- 1 Armstrong's Axioms — reflexivity, augmentation, transitivity + derived rules
- 2 The closure of a functional dependency set, F^+
- 3 Attribute closure, X^+ — the algorithm, and how it finds candidate keys
- 4 Boyce-Codd Normal Form (BCNF) and how it differs from 3NF
- 5 Testing for BCNF using attribute closure
- 6 The classic 3NF-but-not-BCNF relation, and BCNF decomposition

Armstrong's Axioms

Sound (only ever derive FDs that genuinely hold) and **complete** (repeated application derives EVERY implied FD). Working example — StudentAdvisor(StudentNo, StudentName, AdvisorNo, AdvisorName, DeptNo, DeptName):

Axiom	Rule	Example
Reflexivity	If $Y \subseteq X$, then $X \rightarrow Y$ (trivial)	$\{StudentNo, StudentName\} \rightarrow StudentNo$
Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	$StudentNo \rightarrow StudentName$ gives $\{StudentNo, DeptNo\} \rightarrow \{StudentName, DeptNo\}$
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	$StudentNo \rightarrow AdvisorNo, AdvisorNo \rightarrow AdvisorName \Rightarrow StudentNo \rightarrow AdvisorName$

These three are the formal foundation everything else in this lecture builds on.

Derived Rule (from the 3 axioms)	Statement
Union (additivity)	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$
Decomposition (projectivity)	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$ separately
Pseudotransitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$

Closure of FDs (F^+) and Attribute Closure (X^+)

F^+ , the closure of F , is every FD derivable from F by repeatedly applying Armstrong's Axioms — usually far too large to enumerate directly.

NOTE

Attribute closure X^+ (under F) is the set of ALL attributes functionally determined by X — every attribute A such that $X \rightarrow A$ is in F^+ . This answers the one question actually needed over and over: “given X , what does it determine?”

Algorithm:

```
Result := X
repeat
  for each FD (LHS → RHS) in F:
    if LHS ⊆ Result: Result := Result ∪ RHS
until Result stops changing → return Result
```

Finding keys: compute K^+ for a set K ; if K^+ = all attributes of R , K is a superkey. K is a candidate key if it's also MINIMAL. Also tests whether $X \rightarrow Y$ is implied by F : compute X^+ , check $Y \subseteq X^+$.

Computing $\{\text{StudentNo}\}^+$

Start: Result = {StudentNo}

Apply StudentNo $\rightarrow$ StudentName, StudentNo $\rightarrow$ AdvisorNo
Result = {StudentNo, StudentName, AdvisorNo}

Apply AdvisorNo $\rightarrow$ AdvisorName, AdvisorNo $\rightarrow$ DeptNo
AdvisorNo now $\subseteq$ Result (just added)

Apply DeptNo $\rightarrow$ DeptName — done, no FD left to apply
 $\{\text{StudentNo}\}^+ = \text{ALL 6 attributes of StudentAdvisor}$

TIP

$\{\text{StudentNo}\}^+$ is every attribute of StudentAdvisor — so {StudentNo} is a superkey, and trivially minimal (a single attribute) — a CANDIDATE KEY. Checking $\{\text{AdvisorNo}\}^+ = \{\text{AdvisorNo, AdvisorName, DeptNo, DeptName}\}$ — missing StudentNo/StudentName, so NOT a superkey. {StudentNo} is the relation's only candidate key.

Boyce-Codd Normal Form (BCNF)

NOTE

A relation R is in BCNF if, for EVERY non-trivial FD $X \rightarrow Y$ that holds in R, X is a superkey of R — with no exception.

	3NF	BCNF
Exception clause?	Yes, if X isn't a superkey but every attribute of Y is prime	None — X must always be a superkey
Relationship	Every BCNF relation is automatically 3NF	The reverse is NOT guaranteed

Compare directly against 3NF's definition (Lecture 19): BCNF removes 3NF's prime-attribute exception entirely — exactly the gap the next section makes concrete.

```
for each FD (X → Y) in F (or a minimal cover):  
    compute X+; if X+ ≠ all attributes of R → X not a superkey → VIOLATES BCNF
```

Enrollment(StudentNo, CourseNo, InstructorNo)

Business rule: each instructor teaches only one course (in a given term).

StudentNo	CourseNo	InstructorNo
S1	CS201	I1
S2	CS201	I1
S3	CS305	I2

```
F = { {StudentNo, CourseNo} → InstructorNo, InstructorNo → CourseNo }
```

- {StudentNo,CourseNo}+: apply FD1 → adds InstructorNo → all 3 attrs. Superkey, minimal → Candidate key #1
- {StudentNo,InstructorNo}+: apply FD2 (InstructorNo→CourseNo) → adds CourseNo → all 3 attrs. Superkey, minimal → Candidate key #2

Two candidate keys — between them, StudentNo, CourseNo, and InstructorNo are ALL prime attributes.

3NF Passes, BCNF Fails

Is Enrollment in 3NF?

Rule checked	Result
{StudentNo, CourseNo} → InstructorNo	Determinant IS a candidate key ✓
InstructorNo → CourseNo	Determinant not a superkey, but CourseNo is prime — exception applies ✓

Is Enrollment in BCNF?

Rule checked	Result
{StudentNo, CourseNo} → InstructorNo	Determinant IS a candidate key ✓
InstructorNo → CourseNo	Determinant NOT a superkey — VIOLATION, no exception exists

The relation is in 3NF, yet NOT in BCNF — and still carries real redundancy: I1 teaching CS201 repeats once per enrolled student.

- Modification — reassigning I1 to CS410 must update BOTH S1's and S2's rows together
- Insertion — instructor I3 (assigned CS500) can't be recorded until a student enrolls with them
- Deletion — S3 is I2's only enrollment; deleting it erases that I2 teaches CS305 at all

BCNF Decomposition

```

R1 = X ∪ Y           (the violating FD becomes R1's own key)
R2 = (R - Y) ∪ X     (everything else, keeping X to reconnect)

```

Applying it to Enrollment using the violating FD InstructorNo → CourseNo (X={InstructorNo}, Y={CourseNo}):

Teaches (InstructorNo, CourseNo)

InstructorNo	CourseNo
I1	CS201
I2	CS305

Enrolls (StudentNo, InstructorNo)

StudentNo	InstructorNo
S1	I1
S2	I1
S3	I2

TIP

Both are now in BCNF: Teaches's only determinant (InstructorNo) IS its own key. Enrolls has no non-trivial FD left to violate anything. I1's course now lives in exactly one row of Teaches — all three anomalies are gone.

Lossless — But Not Dependency-Preserving

NOTE

A decomposition into R_1 and R_2 is lossless whenever the shared attributes $(R_1 \cap R_2)$ form a key of at least one relation: $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2$ holds in F^+ .

$\text{Teaches} \cap \text{Enrolls} = \{\text{InstructorNo}\}$. InstructorNo is Teaches 's declared PK, so $\text{InstructorNo} \rightarrow \text{Teaches}$ holds trivially. **Lossless: $\text{Teaches} \bowtie \text{Enrolls}$ (on InstructorNo) reconstructs Enrollment exactly.**

WARNING

Check the ORIGINAL FD $\{\text{StudentNo}, \text{CourseNo}\} \rightarrow \text{InstructorNo}$ against the decomposed schema: StudentNo lives only in Enrolls , CourseNo only in Teaches — no single relation has both attributes, so this dependency cannot be verified without joining the tables back together. This is not a mistake — BCNF decomposition always guarantees lossless-join, but does NOT always guarantee dependency-preservation. 3NF (via synthesis, Lecture 21) guarantees both simultaneously — a genuine trade-off.

KEY TAKEAWAYS

Key Takeaways

- 1 Armstrong's Axioms — reflexivity, augmentation, transitivity — are sound and complete; union, decomposition, pseudotransitivity are convenient derived rules
- 2 Attribute closure (X^+) finds every attribute X determines, and is the mechanical way to find candidate keys
- 3 BCNF requires every non-trivial FD's determinant to be a superkey — no exception. 3NF tolerates one: a non-superkey determinant is OK if every attribute it determines is prime
- 4 Enrollment(StudentNo, CourseNo, InstructorNo) with $\text{InstructorNo} \rightarrow \text{CourseNo}$ is 3NF but NOT BCNF — and still carries real redundancy
- 5 BCNF decomposition splits along a violating FD ($R_1=XY$, $R_2=(R-Y)UX$) and is always lossless-join, but can sacrifice dependency-preservation